

Politechnika Rzeszowska
Wydział Matematyki i Fizyki Stosowanej
Kierunek: Inżynieria i Analiza Danych

Sprawozdanie

Algorytmy i struktury danych

ZADANIE PROJEKTOWE/ALGORYTMICZNE

Autor:
MATEUSZ DZIAŁOWSKI

Nr indeksu:
184213

Prowadzący:
dr inż. prof. PRz Mariusz Borkowski

26 stycznia 2026

Spis treści

1	Wstęp	2
2	Opis problemu	3
3	Podstawy teoretyczne	3
3.1	Teoretyczna złożoność algorytmu	3
4	Szczegóły implementacji	3
4.1	Rozwiązanie problemu - podejście pierwsze	3
4.1.1	Analiza problemu	3
4.1.2	Schemat blokowy algorytmu	5
4.1.3	Algorytm zapisywany w pseudokodzie	6
4.1.4	Sprawdzenie poprawności algorytmu	7
4.1.5	Teoretyczne oszacowanie złożoności obliczeniowej	7
4.2	Rozwiązanie problemu - podejście drugie	7
4.2.1	Schemat blokowy algorytmu	8
4.2.2	Algorytm zapisywany w pseudokodzie	9
4.2.3	Sprawdzenie poprawności algorytmu	9
4.2.4	Teoretyczne oszacowanie złożoności obliczeniowej	10
4.3	Implementacja algorytmów w języku C++	10
4.3.1	Implementacja	10
4.3.2	Testy niewygodnych zestawów danych	12
4.3.3	Test wydajnościowy algorytmów	15
5	Podsumowanie	20
	Aneks: Kod programu	21

1 Wstęp

Celem niniejszego opracowania projektu jest przedstawienie implementacji oraz analizy wydajnościowej algorytmu, rozwiązującego problem wyszukiwania specyficznych podciągów w zadanym ciągu liczb całkowitych dodatnich wejściowych. Dokumentacja obejmuje szczegółowy opis problemu, podstawy teoretyczne, implementację algorytmu oraz wyniki testów wydajnościowych przeprowadzonych na różnych zestawach danych wejściowych.

W przedstawionym opracowaniu indeksowanie pozycji w podciągach i tablicach zaczyna się od 0.

2 Opis problemu

Dla danego ciągu liczb całkowitych dodatnich $A[]$ o długości N , należy znaleźć wszystkie **spójne** podciągi o długości $M = 5$ (pięcioelementowe), które spełniają następujące kryterium: suma elementów na pozycjach nieparzystych (indeksy 1 i 3) jest większa od sumy elementów na pozycjach parzystych (indeksy 0, 2 i 4). Przez **spójny podciąg** rozumiemy ciąg kolejnych, sąsiadujących ze sobą elementów w tablicy wejściowej.

Dla podciągu $T[] = [t_0, t_1, t_2, t_3, t_4]$, warunek przyjmuje postać:

$$t_1 + t_3 > t_0 + t_2 + t_4 \quad (1)$$

Przykład: Dla ciągu $A[] = [1, 130, 1, 9, 11, 6, 1, 1, 1, 3, 1]$, zgodnie z powyższym kryterium, spójne podciągi spełniające warunek to:

- $T[M] = [1, 130, 1, 9, 11]$, ponieważ $(130 + 9) > (1 + 1 + 11) \Rightarrow 139 > 13$
- $T[M] = [1, 9, 11, 6, 1]$, ponieważ $(9 + 6) > (1 + 11 + 1) \Rightarrow 15 > 13$
- $T[M] = [1, 1, 1, 3, 1]$, ponieważ $(1 + 3) > (1 + 1 + 1) \Rightarrow 4 > 3$

Jak widać, w powyższym przykładzie istnieją trzy spójne podciągi spełniające zadane kryterium.

3 Podstawy teoretyczne

Algorytm opiera się na technice przesuwania okna o stałej $M = 5$. Dla każdego możliwego podciągu o długości M w ciągu $A[]$, algorytm sprawdza, czy spełnia on warunek określony w równaniu (1). Jeśli tak, podciąg jest zapisywany do wyniku.

3.1 Teoretyczna złożoność algorytmu

Teoretyczna analiza złożoności algorytmu przedstawia się następująco:

- **Złożoność czasowa:** Sprawdzenie jednego okna zajmuje czas stały $\mathcal{O}(1)$. Ponieważ okno przesuwamy przez cały ciąg o długości N , wykonujemy $N - M + 1$ operacji. M jest stałą, co powoduje, że teoretyczna złożoność powinna wynosić $\mathcal{O}(N)$.
- **Złożoność pamięciowa:** Algorytm wymaga przechowywania ciągu wejściowego o wielkości N oraz tablicy wyników, która jest alokowana dynamicznie. Teoretycznie, w najgorszym przypadku, gdy każdy podciąg spełnia warunek, złożoność pamięciowa wynosi $\mathcal{O}(N)$.

4 Szczegóły implementacji

4.1 Rozwiązanie problemu - podejście pierwsze

4.1.1 Analiza problemu

Choć algorytm jest stosunkowo prosty, jego implementacja wymaga uwzględnienia kilku kluczowych aspektów, takich jak efektywne zarządzanie pamięcią, optymalne podejście przy wykorzystaniu pętli

oraz optymalizacja operacji sprawdzania warunków, co może znacząco wpłynąć na wydajność końcowego rozwiązania. Pod uwagę trzeba wziąć również obsługę różnych przypadków brzegowych, takich jak bardzo krótkie ciągi wejściowe lub ciągi niespełniające żadnego z kryteriów, co wciąż wpływa na wydajność implementacji.

W podejściu pierwszym postąpimy w sposób bezpośredni, iterując przez każdy możliwy podciąg o długości M w ciągu $A[N]$ i sprawdzając, czy spełnia on warunek określony w równaniu (1). Jeśli tak, dodajemy go do listy wyników. Liście wyników nie przypisujemy żadnej wartości elementowej, ponieważ przyjmujemy, że tablica dynamicznie zmienia swoje rozmiary. Również musimy wziąć pod uwagę przypadki, gdy długość ciągu N jest mniejsza niż M lub dane wejściowe są ułożone w sposób uniemożliwiający znalezienie podciągu spełniającego warunek. W takim przypadku nie ma możliwości znalezienia żadnego podciągu o wymaganej długości, więc algorytm powinien zwrócić pustą listę wyników.

1. Dane wejściowe.

Naszymi danymi wejściowymi są:

- Ciąg liczb całkowitych dodatnich $Tab[]$,
- długość ciągu N ,
- stała $M = 5$.

2. Dane wyjściowe.

Naszymi danymi wyjściowymi są:

- Tablica wyników $Wynik[]$, zawierająca wszystkie podciągi.

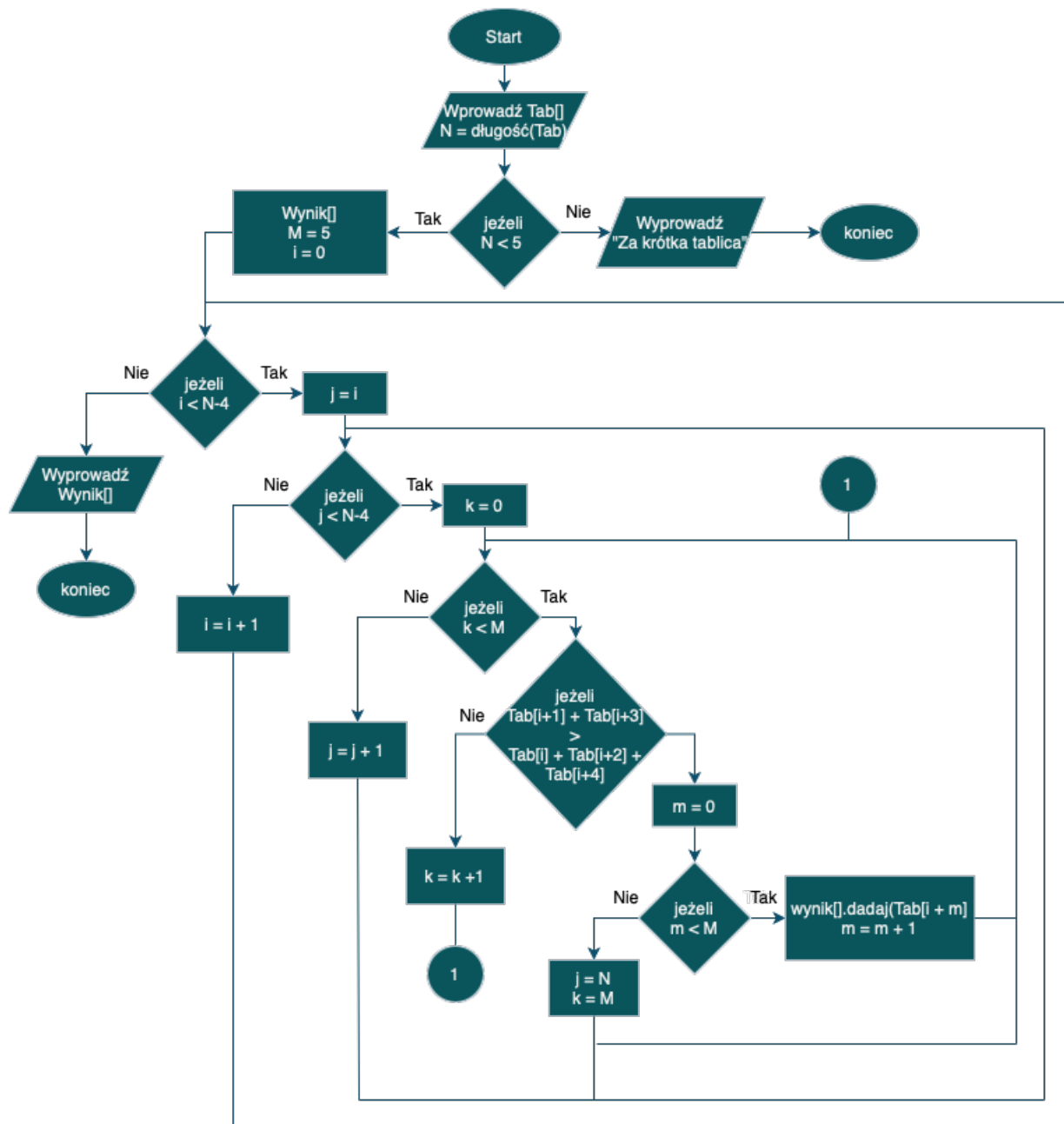
Implementacja wykorzystuje zagnieżdżone pętle iteracyjne (warunki zapętłone) wraz ze zmiennymi sterującymi, aby efektywnie przeszukiwać ciąg wejściowy i identyfikować podciągi spełniające określone kryteria. Poniżej przedstawiono szczegółowy opis działania poszczególnych pętli:

- **Pętla zewnętrzna $i < N-4$ (i)** - przesuwa okno przez cały ciąg (od 0 do $N-4$). Ze względu na warunek sprawdzający (1) musimy pętlę ograniczyć do $N-4$, w celu uniknięcia błędów. W przedstawionej pętli sprawdzamy krok po kroku każdy możliwy podciąg o długości $M = 5$, czy spełnia on warunek z równania (1). Pętla przesuwa okno o jeden element do przodu w każdej iteracji w celu sprawdzenia następnych możliwych podciągów.
- **Pętla wewnętrzna $j < N-4$ (j)** - rozpoczyna się od pozycji $j = i$ i iteruje do $N-4$, sprawdzając warunek z równania (1) dla każdego podciągu zaczynającego się na pozycji i .
- **Pętla iteracyjna $k < M$ (k)** - przechodzi przez elementy podciągu (od 0 do $M-1$), aby obliczyć sumy wymagane do sprawdzenia warunku z równania (1). W tej pętli sumujemy odpowiednie elementy podciągu, aby porównać je zgodnie z kryterium określonym w równaniu (1).
- **Pętla kopiująca $m < M$ (m)** - przepisuje spełniający warunek podciąg do tablicy wyników $Wynik[]$. Ta pętla iteruje przez elementy podciągu i kopiuje je do tablicy wyników, gdy warunek z równania (1) jest spełniony.

Gdy warunek z równania (1) zostanie spełniony dla danego okna, algorytm kopiuje 5 elementów do tablicy wyników i przerywa wewnętrzne pętle poprzez ustawienie wartości sterujących ($j = N$, $k = M$), co wymusza przejście do następnej pozycji okna.

4.1.2 Schemat blokowy algorytmu

Poniżej przedstawiono schemat blokowy ilustrujący działanie algorytmu:



Rys. 1: Schemat blokowy algorytmu wyszukiwania podciągów

4.1.3 Algorytm zapisywany w pseudokodzie

W poniższym pseudokodzie przedstawiono szczegółową implementację algorytmu. W przypadku pseudokodu przyjęliśmy wykorzystanie pętli `while` do iteracji przez elementy ciągu oraz do sprawdzania warunków, w porównaniu do schematu blokowego, gdzie przyjęliśmy warunki jako pętle.

W pseudokodzie użyto następujących oznaczeń:

- `Tab[]` - tablica wejściowa zawierająca ciąg liczb całkowitych dodatnich,
- `N` - długość tablicy wejściowej,
- `wynik[]` - tablica wynikowa przechowująca znalezione podciągi,
- `M` - stała określająca długość podciągu ($M = 5$),
- `i`, `j`, `k`, `m` - zmienne sterujące pętlą.

```
Input  : Tab[], N
Output: wynik[] lub -1
if  $N < 5$  then
  return -1
wynik  $\leftarrow []$ ;
const  $M \leftarrow 5$ ;
 $i \leftarrow 0$ ;
while  $i < N - 4$  do
   $j \leftarrow i$ ;
  while  $j < N - 4$  do
     $k \leftarrow 0$ ;
    while  $k < M$  do
      if  $Tab[i + 1] + Tab[i + 3] > Tab[i] + Tab[i + 2] + Tab[i + 4]$  then
         $m \leftarrow 0$ ;
        while  $m < M$  do
          wynik.dodaj( $Tab[i + m]$ );
           $m \leftarrow m + 1$ ;
         $j \leftarrow N$ ;
         $k \leftarrow M$ ;
      else
         $k \leftarrow k + 1$ ;
     $j \leftarrow j + 1$ ;
   $i \leftarrow i + 1$ ;
return wynik;
```

Jak widać, algorytm iteruje przez każdy możliwy podciąg o długości M w ciągu `Tab[]`, sprawdzając, czy spełnia on warunek określony w równaniu (1). Jeśli tak, podciąg jest dodawany do tablicy wyników `wynik[]`. W przypadku, gdy długość ciągu N jest mniejsza niż M , algorytm zwraca -1 , wskazując na brak możliwości znalezienia podciągu spełniającego warunek.

4.1.4 Sprawdzenie poprawności algorytmu

Aby zweryfikować poprawność działania zaimplementowanego algorytmu, przede wszystkim przeprowadzimy test "ołówkowy".

Dla przykładowego ciągu wejściowego $A[] = [5, 20, 5, 30, 5, 2, 1, 2, 1]$

- Długość ciągu $N = 9$,
- Stała $M = 5$.

Krok	i	j	k	m	Sprawdzany warunek	Wynik / Akcja
1	0	0	0	-	$(20 + 30) > (5 + 5 + 5) \rightarrow 50 > 15$	Prawda
2	0	0	0	0-4	Kopiowanie Tab[0...4]	Dodano: [5, 20, 5, 30, 5]
3	0	9	5	-	-	Ustawienie $j = N$, $k = M$ (koniec pętli dla $i = 0$)
4	1	1	0	-	$(5 + 5) > (20 + 30 + 2) \rightarrow 10 > 52$	Fałsz
5	1	1	1	-	-	k zwiększa się o 1
...	1	...	...	-	-	k dochodzi do 5, potem j do $N - 4$
6	2	2	0	-	$(30 + 2) > (5 + 5 + 1) \rightarrow 32 > 11$	Prawda
7	2	2	0	0-4	Kopiowanie Tab[2...6]	Dodano: [5, 30, 5, 2, 1]
8	2	9	5	-	-	Koniec pętli dla $i = 2$
9	3	3	0	-	$(5 + 1) > (30 + 2 + 2) \rightarrow 6 > 34$	Fałsz
10	4	4	0	-	$(2 + 2) > (5 + 1 + 1) \rightarrow 4 > 7$	Fałsz

Tabela 1: Przebieg testu ołówkowego dla ciągu $A[]$

4.1.5 Teoretyczne oszacowanie złożoności obliczeniowej

Analiza złożoności algorytmu przedstawia się następująco:

W algorytmie pętla i oraz j są zależne od N , natomiast pętla k i m mają stałą liczbę przebiegów ($M = 5$), więc wnoszą stały koszt. Liczbę porównań w gałęzi `if` wyznacza podwójna pętla:

- i przebiega od 0 do $N - 5 \Rightarrow$ około N iteracji,
- dla każdego i zmienna j przechodzi od i do $N - 5 \Rightarrow (N - 4 - i)$ iteracji.

Łączna liczba porównań to więc suma

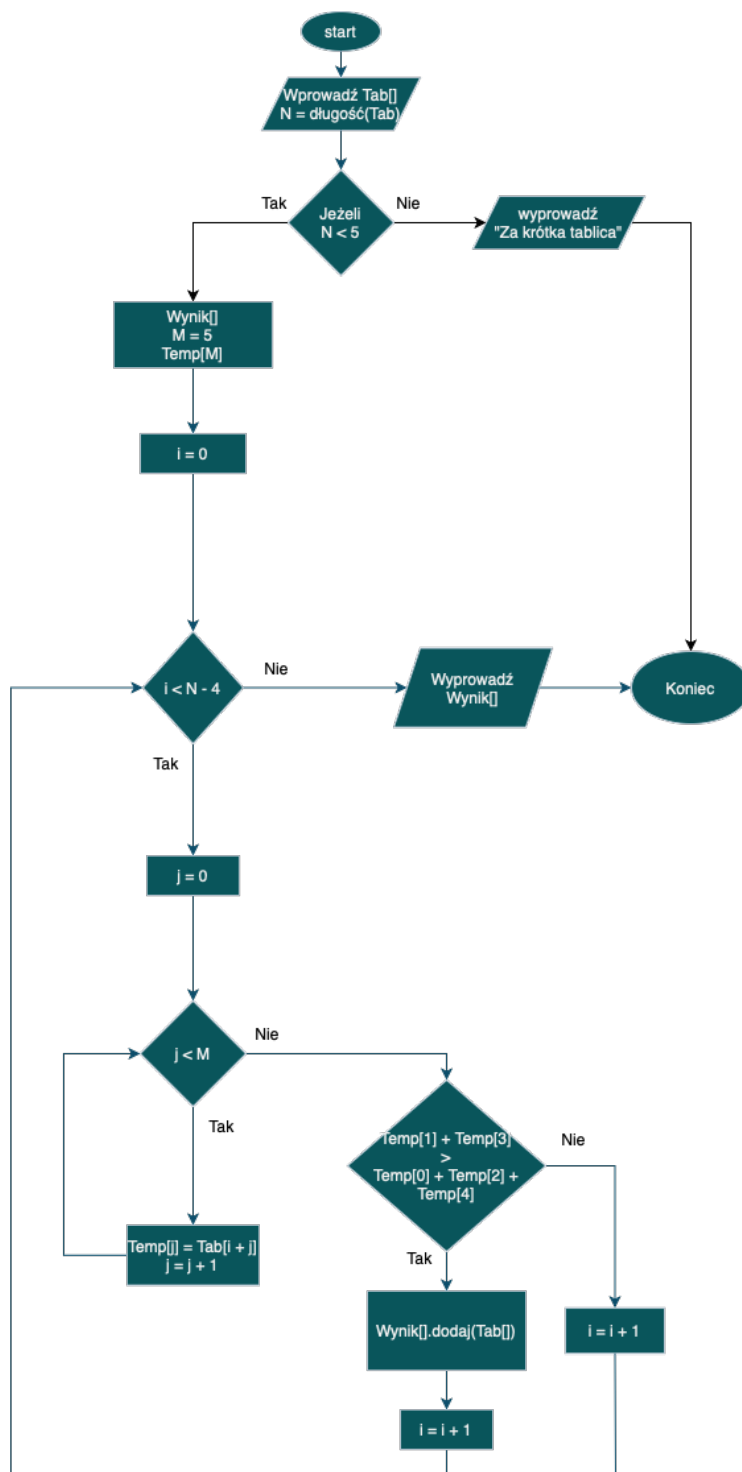
$$\sum_{i=0}^{N-5} (N - 4 - i) = 1 + 2 + \dots + (N - 4) = \frac{(N - 4)(N - 3)}{2} = \mathcal{O}(N^2). \quad (2)$$

Pętla k i m są stałej długości, więc nie zmieniają rzędu złożoności. Całkowita złożoność czasowa algorytmu wynosi zatem $\mathcal{O}(N^2)$.

4.2 Rozwiązanie problemu - podejście drugie

Po przeanalizowaniu pierwszego podejścia zauważamy, że jego złożoność czasowa wynosi $\mathcal{O}(N^2)$, co może być nieefektywne dla dużych wartości N . W związku z tym proponujemy drugie podejście, które optymalizuje algorytm do złożoności $\mathcal{O}(N)$ poprzez eliminację zbędnych pętli.

4.2.1 Schemat blokowy algorytmu



Rys. 2: Schemat blokowy algorytmu wyszukiwania podciągów

4.2.2 Algorytm zapisywany w pseudokodzie

Poniżej przedstawiono szczegółową implementację zoptymalizowanego algorytmu.

W pseudokodzie użyto następujących oznaczeń:

- $\text{Tab}[]$ - tablica wejściowa zawierająca ciąg liczb całkowitych dodatnich,
- N - długość tablicy wejściowej,
- $\text{wynik}[]$ - tablica wynikowa przechowująca znalezione podciągi,
- M - stała określająca długość podciągu ($M = 5$),
- i, j - zmienne sterujące pętli.

```
Input  :  $\text{Tab}[], N$ 
Output:  $\text{wynik}[]$  lub  $-1$ 
if  $N < 5$  then
  | return  $-1$ 
 $\text{wynik} \leftarrow []$ ;
const  $M \leftarrow 5$ ;
 $\text{Temp}[M]$ ;
 $i \leftarrow 0$ ;
while  $i < N - 4$  do
  |  $j \leftarrow 0$ ;
  | while  $j < M$  do
  | |  $\text{Temp}[j] \leftarrow \text{Tab}[i + j]$ ;
  | |  $j \leftarrow j + 1$ ;
  | if  $\text{Temp}[1] + \text{Temp}[3] > \text{Temp}[0] + \text{Temp}[2] + \text{Temp}[4]$  then
  | |  $\text{wynik.dodaj}(\text{Temp}[])$ ;
  |  $i \leftarrow i + 1$ ;
return  $\text{wynik}$ ;
```

4.2.3 Sprawdzenie poprawności algorytmu

Aby zweryfikować poprawność działania zaimplementowanego algorytmu, również przeprowadzimy test "ołówkowy".

Dla przykładowego ciągu wejściowego $A[] = [5, 20, 5, 30, 5, 2, 1, 2, 1]$

- Długość ciągu $N = 9$,
- Stała $M = 5$.

Krok	i	j	Temp[]	Sprawdzany warunek	Wynik / Akcja
1	0	0-4	[5, 20, 5, 30, 5]	$(20+30) > (5+5+5) \rightarrow 50 > 15$	Prawda
2	0	-	-	-	Dodano: [5, 20, 5, 30, 5]
3	1	0-4	[20, 5, 30, 5, 2]	$(5+5) > (20+30+2) \rightarrow 10 > 52$	Fałsz
4	2	0-4	[5, 30, 5, 2, 1]	$(30+2) > (5+5+1) \rightarrow 32 > 11$	Prawda
5	2	-	-	-	Dodano: [5, 30, 5, 2, 1]
6	3	0-4	[30, 5, 2, 1, 2]	$(5+1) > (30+2+2) \rightarrow 6 > 34$	Fałsz
7	4	0-4	[5, 2, 1, 2, 1]	$(2+2) > (5+1+1) \rightarrow 4 > 7$	Fałsz

Tabela 2: Przebieg testu ołówkowego dla ciągu $A[]$

4.2.4 Teoretyczne oszacowanie złożoności obliczeniowej

Analiza złożoności algorytmu przedstawia się następująco: W algorytmie pętla i jest zależna od N , natomiast pętla j ma stałą liczbę przebiegów ($M = 5$), więc wnosi stały koszt. Liczbę porównań w gałęzi `if` wyznacza pojedyncza pętla:

- i przebiega od 0 do $N - 4 \Rightarrow$ oznacza to około N iteracji,

Łączna liczba porównań to więc około N . Pętla j jest stałej długości, więc nie zmienia rzędu złożoności. Całkowita złożoność czasowa algorytmu wynosi zatem $\mathcal{O}(N)$.

4.3 Implementacja algorytmów w języku C++

4.3.1 Implementacja

Implementacja obu podejść została przeprowadzona w języku C++. Poniżej przedstawiono kluczowe fragmenty kodu dla obu algorytmów:

```

1 std::vector<int> podciag(int ciag[], int N) {
2     if (N < 5) {
3         return {};
4     }
5     const int M = 5;
6     std::vector<int> wynik;
7     for(int i = 0; i < N - 4; i++) {
8         for(int j = i; j < N - 4; j++) {
9             for(int k = 0; k < M; k++) {
10                if(ciag[i+1] + ciag[i+3] > ciag[i] + ciag[i+2] + ciag[i+4]) {
11                    for(int m = 0; m < M; m++) {
12                        wynik.push_back(ciag[i + m]);
13                    }
14                    j = N;
15                    k = M;
16                    break;
17                }
18            }
19        }
20    }

```

```

21     return wynik;
22 }
23
24 std::vector<int> podciag_wersja_2(int ciag[], int N) {
25     if (N < 5) {
26         return {};
27     }
28     const int M = 5;
29     int Temp[M];
30     std::vector<int> wynik;
31     for(int i = 0; i < N - 4; i++) {
32         for(int j = 0; j < M; j++) {
33             Temp[j] = ciag[i + j];
34         }
35         if(Temp[1] + Temp[3] > Temp[0] + Temp[2] + Temp[4]) {
36             for(int k = 0; k < M; k++) {
37                 wynik.push_back(Temp[k]);
38             }
39         }
40     }
41     return wynik;
42 }
43
44 int main() {
45     int tab[] = {1, 130, 1, 9, 11, 6, 1, 1, 1, 3, 1};
46     int N = sizeof(tab)/sizeof(tab[0]);
47     std::vector<int> wynik = podciag(tab, N);
48     wyswietl_wynik(wynik);
49     std::vector<int> wynik2 = podciag_wersja_2(tab, N);
50     wyswietl_wynik(wynik2);
51     return 0;
52 }

```

Jak widać, oba podejścia implementują funkcje `podciag` oraz `podciag_wersja_2`, które realizują opisane algorytmy i zwracają odpowiednie podciągi spełniające warunki.

4.3.2 Testy niewygodnych zestawów danych

Aby ocenić poprawność zaimplementowanych algorytmów, przeprowadzono testy na różnych zestawach danych wejściowych. Poniżej przedstawiono kod źródłowy testów:

```
1 std::vector<int> podciag(int ciag[], int N) {
2     if (N < 5) {
3         return {};
4     }
5     const int M = 5;
6     std::vector<int> wynik;
7     for(int i = 0; i < N - 4; i++) {
8         for(int j = i; j < N - 4; j++) {
9             for(int k = 0; k < M; k++) {
10                if(ciag[i+1] + ciag[i+3] > ciag[i] + ciag[i+2] + ciag[i+4]) {
11                    for(int m = 0; m < M; m++) {
12                        wynik.push_back(ciag[i + m]);
13                    }
14                    j = N;
15                    k = M;
16                    break;
17                }
18            }
19        }
20    }
21    return wynik;
22 }
23
24 std::vector<int> podciag_wersja_2(int ciag[], int N) {
25     if (N < 5) {
26         return {};
27     }
28     const int M = 5;
29     int Temp[M];
30     std::vector<int> wynik;
31     for(int i = 0; i < N - 4; i++) {
32         for(int j = 0; j < M; j++) {
33             Temp[j] = ciag[i + j];
34         }
35         if(Temp[1] + Temp[3] > Temp[0] + Temp[2] + Temp[4]) {
36             for(int k = 0; k < M; k++) {
37                 wynik.push_back(Temp[k]);
38             }
39         }
40     }
41     return wynik;
42 }
43
44 void wyswietl_dane(int tablica[], int N) {
45     std::cout << "Ciag wejsciowy: ";
46     for (int i = 0; i < N; i++) {
47         std::cout << tablica[i] << " ";
```

```

48     }
49     std::cout << std::endl;
50 }
51
52 void wyswietl_wynik(std::vector<int> wynik) {
53     if(!wynik.empty()) {
54         int k{0};
55         for (int i = 0; i < wynik.size() / 5; i++) {
56             std::cout << "[ ";
57             for (int j = 0; j < 5; j++) {
58                 std::cout << wynik[j+k] << " ";
59             }
60             std::cout << "]";
61             if (i < wynik.size() / 5 - 1) std::cout << ", ";
62             k += 5;
63         }
64         std::cout << std::endl;
65     }
66     else {
67         std::cout << "Ciag jest za krotki lub nie ma zadnego
68         spelnionego ciagu\n";
69     }
70 }
71 struct Wynik {
72     std::vector<int> p1;
73     std::vector<int> p2;
74 };
75
76 void test_niewygodnych_zestawow() {
77     int tab1[] = {1, 3, 2, 3};
78     int tab2[] = {7, 7, 7, 7, 7};
79     int tab3[] = {1, 130, 1, 9, 11, 6, 1, 1, 1, 3, 1};
80     int tab4[] = {1, 100, 1, 100, 1, 2, 2, 2, 2};
81     int tab5[] = {5, 200, 5, 200, 5, 10, 10, 10, 10};
82
83     struct Test {
84         int* tablica;
85         int rozmiar;
86     };
87
88     Test testy[] = {
89         {tab1, sizeof(tab1)/sizeof(tab1[0])},
90         {tab2, sizeof(tab2)/sizeof(tab2[0])},
91         {tab3, sizeof(tab3)/sizeof(tab3[0])},
92         {tab4, sizeof(tab4)/sizeof(tab4[0])},
93         {tab5, sizeof(tab5)/sizeof(tab5[0])}
94     };
95
96     for (int i = 0; i < 5; i++) {
97         std::cout << "\n=== TEST " << (i+1) << " ===" << std::endl;

```

```

98     std::cout << "Wejście: ";
99     std::cout << std::endl;
100    wyswietl_dane(testy[i].tablica, testy[i].rozmiar);
101
102    Wynik wynik = test_zestawow(
103        std::vector<int>(testy[i].tablica, testy[i].tablica +
104        testy[i].rozmiar),
105        testy[i].rozmiar
106    );
107
108    std::cout << std::endl;
109    std::cout << "Algorytm 1: ";
110    wyswietl_wynik(wynik.p1);
111    std::cout << "Algorytm 2: ";
112    wyswietl_wynik(wynik.p2);
113 }
114
115 int main() {
116     test_niewygodnych_zestawow();
117     return 0;
118 }

```

Wyniki testów:

=== TEST 1 ===

Wejście:

Ciag wejsciowy: 1 3 2 3

Algorytm 1: Ciag jest za krotki lub nie ma zadnego spelnionego ciagu

Algorytm 2: Ciag jest za krotki lub nie ma zadnego spelnionego ciagu

=== TEST 2 ===

Wejście:

Ciag wejsciowy: 7 7 7 7 7

Algorytm 1: Ciag jest za krotki lub nie ma zadnego spelnionego ciagu

Algorytm 2: Ciag jest za krotki lub nie ma zadnego spelnionego ciagu

=== TEST 3 ===

Wejście:

Ciag wejsciowy: 1 130 1 9 11 6 1 1 1 3 1

Algorytm 1: [1 130 1 9 11], [1 9 11 6 1], [1 1 1 3 1]

Algorytm 2: [1 130 1 9 11], [1 9 11 6 1], [1 1 1 3 1]

=== TEST 4 ===

Wejście:

Ciag wejsciowy: 1 100 1 100 1 2 2 2 2

Algorytm 1: [1 100 1 100 1], [1 100 1 2 2]

Algorytm 2: [1 100 1 100 1], [1 100 1 2 2]

=== TEST 5 ===

Wejście:

Ciag wejsciowy: 5 200 5 200 5 10 10 10 10

Algorytm 1: [5 200 5 200 5], [5 200 5 10 10]

Algorytm 2: [5 200 5 200 5], [5 200 5 10 10]

4.3.3 Test wydajnościowy algorytmów

```
1
2 std::vector<int> podciag(int ciag[], int N) {
3     if (N < 5) {
4         return {};
5     }
6     const int M = 5;
7     std::vector<int> wynik;
8     for(int i = 0; i < N - 4; i++) {
9         for(int j = i; j < N - 4; j++) {
10            for(int k = 0; k < M; k++) {
```



```

11         if(ciang[i+1] + ciag[i+3] > ciag[i] + ciag[i+2] + ciag
    [i+4]) {
12             for(int m = 0; m < M; m++) {
13                 wynik.push_back(ciang[i + m]);
14             }
15             j = N;
16             k = M;
17             break;
18         }
19     }
20 }
21 }
22 return wynik;
23 }
24
25 std::vector<int> podciag_wersja_2(int ciag[], int N) {
26     if (N < 5) {
27         return {};
28     }
29     const int M = 5;
30     int Temp[M];
31     std::vector<int> wynik;
32     for(int i = 0; i < N - 4; i++) {
33         for(int j = 0; j < M; j++) {
34             Temp[j] = ciag[i + j];
35         }
36         if(Temp[1] + Temp[3] > Temp[0] + Temp[2] + Temp[4]) {
37             for(int k = 0; k < M; k++) {
38                 wynik.push_back(Temp[k]);
39             }
40         }
41     }
42     return wynik;
43 }
44
45 void generuj_dane(int ciag[], int N) {
46     std::srand(std::time(NULL));
47     for (int i = 0; i < N; i++) {
48         ciag[i] = rand() % 150 + 1; // rand() % (max - min + 1) +
    min
49     }
50 }
51
52 void wyswietl_dane(int tablica[], int N) {
53     std::cout << "Ciag wejsciowy: ";
54     for (int i = 0; i < N; i++) {
55         std::cout << tablica[i] << " ";
56     }
57     std::cout << std::endl;
58 }
59

```

```

60 void wyswietl_wynik(std::vector<int> wynik) {
61     if(!wynik.empty()) {
62         int k{0};
63         for (int i = 0; i < wynik.size() / 5; i++) {
64             std::cout << "[";
65             for (int j = 0; j < 5; j++) {
66                 std::cout << wynik[j+k] << " ";
67             }
68             std::cout << "];";
69             if (i < wynik.size() / 5 - 1) std::cout << ", ";
70             k += 5;
71         }
72         std::cout << std::endl;
73     }
74     else {
75         std::cout << "Ciag jest za krotki lub nie ma zadnego
76         spelnionego ciagu\n";
77     }
78 }
79 void zapis_do_pliku(std::vector<int> wynik, const std::string&
80     filename) {
81     std::ofstream file(filename);
82     if (file.is_open()) {
83         if(!wynik.empty()) {
84             int k{0};
85             for (int i = 0; i < wynik.size() / 5; i++) {
86                 file << "[";
87                 for (int j = 0; j < 5; j++) {
88                     file << wynik[j+k] << " ";
89                 }
90                 file << "];";
91                 if (i < wynik.size() / 5 - 1) file << ", ";
92                 k += 5;
93             }
94         }
95     }
96 }
97 struct Wynik {
98     std::vector<int> p1;
99     std::vector<int> p2;
100 };
101
102 Wynik test_zestawow(std::vector<int> tab, int N) {
103     std::vector<int> wynik = podciag(tab.data(), N);
104     std::vector<int> wynik2 = podciag_wersja_2(tab.data(), N);
105     return {wynik, wynik2};
106 }
107
108 void test_wydajnosci() {

```

```

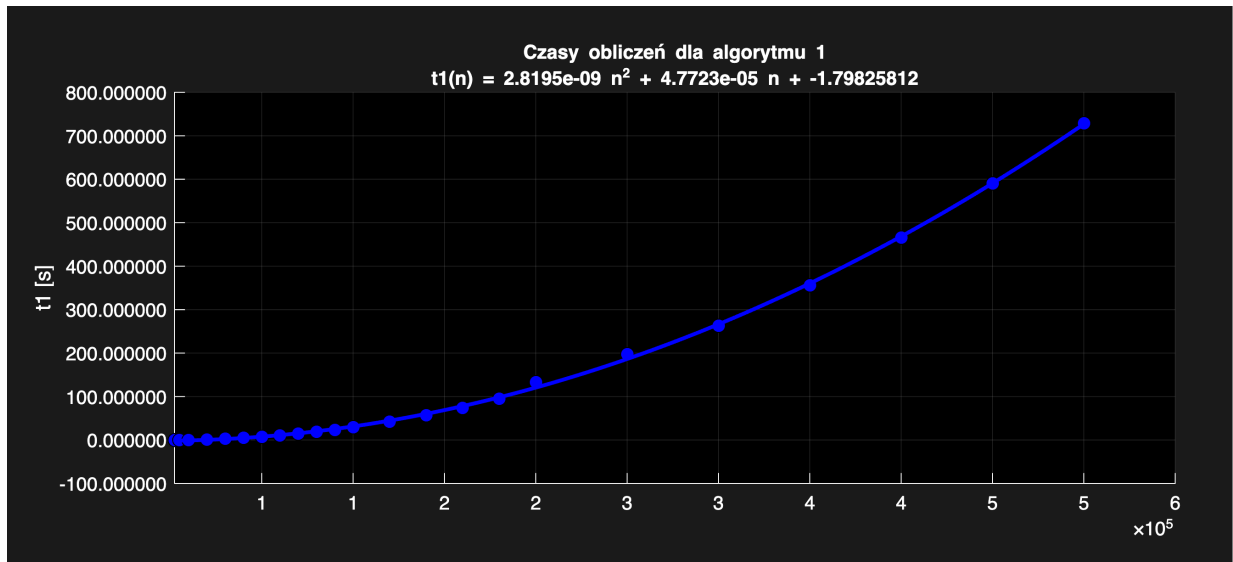
109     int N[] = {2500, 5000, 10000, 20000, 30000, 40000, 50000, 60000,
110             70000, 80000};
111     int liczba_testow = sizeof(N)/sizeof(N[0]);
112     double* czas1 = new double[liczba_testow];
113     double* czas2 = new double[liczba_testow];
114
115     for(int i = 0; i < liczba_testow; i++) {
116         std::vector<int> ciag_wejscowy(N[i]);
117         std::vector<int> wynik(N[i]);
118
119         generuj_dane(ciag_wejscowy.data(), N[i]);
120
121         clock_t start1 = clock();
122         std::vector<int> wynik1 = podciag(ciag_wejscowy.data(), N[i]);
123         clock_t stop1 = clock();
124         czas1[i] = (double)(stop1 - start1) / CLOCKS_PER_SEC;
125
126         clock_t start2 = clock();
127         std::vector<int> wynik2 = podciag_wersja_2(ciag_wejscowy.data(), N[i]);
128         clock_t stop2 = clock();
129         czas2[i] = (double)(stop2 - start2) / CLOCKS_PER_SEC;
130     }
131     std::cout << "L.p.   n   t1[s]   t2[s]" << "\n";
132     for(int i = 0; i < liczba_testow; i++) {
133         std::cout << std::fixed << std::setprecision(6) << i+1 << "
134         " << N[i] << "   " << czas1[i] << "   " << czas2[i] << '\n';
135     }
136
137     std::string filename = "wyniki_wydajnosci.txt";
138     std::ofstream file;
139     file.open("wyniki_wydajnosci.txt");
140     file << "L.p.   n   t1[s]   t2[s]" << "\n";
141     for(int i = 0; i < liczba_testow; i++) {
142         file << std::fixed << std::setprecision(6) << i+1 << "   " <<
143         N[i] << "   " << czas1[i] << "   " << czas2[i] << '\n';
144     }
145     file.close();
146
147     delete[] czas1;
148     delete[] czas2;
149
150 int main () {
151     test_wydajnosci();
152
153     return 0;
154 }

```

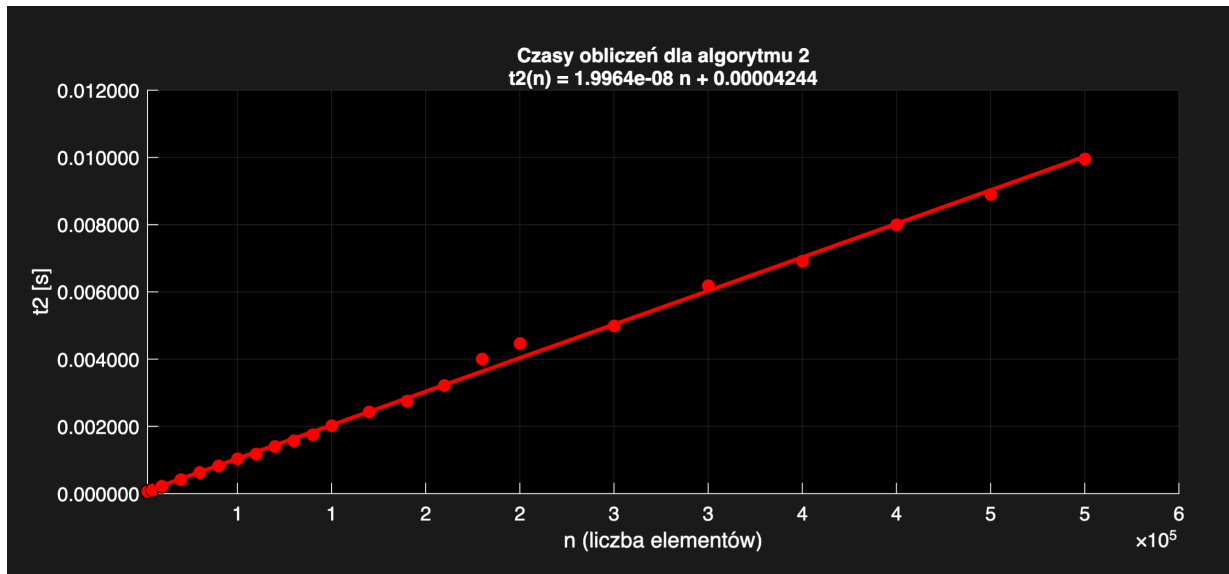
Wyniki testów wydajnościowych przedstawiono w poniższej tabeli:

L.p.	n	t_1 [s]	t_2 [s]
1	2500	0.017936	0.000058
2	5000	0.072735	0.000110
3	10000	0.286721	0.000221
4	20000	1.140572	0.000415
5	30000	2.574637	0.000622
6	40000	4.592503	0.000826
7	50000	7.236926	0.001031
8	60000	10.379715	0.001175
9	70000	14.244029	0.001394
10	80000	18.563128	0.001575
11	90000	23.476461	0.001755
12	100000	28.936148	0.002018
13	120000	41.625615	0.002428
14	140000	56.954450	0.002748
15	160000	74.147522	0.003218
16	180000	94.896126	0.003997
17	200000	133.425706	0.004472
18	250000	197.493877	0.004983
19	300000	262.953962	0.006182
20	350000	355.814712	0.006918
21	400000	465.987660	0.008000
22	450000	590.600902	0.008892
23	500000	728.519425	0.009960

Tabela 3: Wyniki testów wydajnościowych



Rys. 3: Wykres wydajności algorytmu 1 - zależność czasu wykonania od rozmiaru danych



Rys. 4: Wykres wydajności algorytmu 2 - zależność czasu wykonania od rozmiaru danych

5 Podsumowanie

W niniejszym sprawozdaniu przedstawiono dwa podejścia do rozwiązania problemu wyszukiwania podciągów w ciągu liczb całkowitych dodatnich, spełniających określony warunek. Pierwsze podejście, choć poprawne, charakteryzuje się złożonością czasową $\mathcal{O}(N^2)$, co może być nieefektywne dla dużych wartości N . Drugie podejście optymalizuje algorytm do złożoności $\mathcal{O}(N)$, eliminując zbędne pętle i poprawiając wydajność.

Testy na niewygodnych zestawach danych potwierdziły poprawność działania obu podejść, a testy wydajnościowe wykazały znaczną przewagę drugiego podejścia pod względem czasu wykonania.

Wyniki testów wydajnościowych jednoznacznie wskazują na efektywność drugiego podejścia, co czyni je preferowanym rozwiązaniem dla problemu wyszukiwania podciągów w dużych ciągach danych.

Aneks: Kod programu

Główny program (C++)

```
1 #include <iostream>
2 #include <vector>
3 #include <cstdlib>
4 #include <ctime>
5 #include <iomanip>
6 #include <fstream>
7
8 std::vector<int> podciag(int ciag[], int N) {
9     //program kończy się, jeśli tablica ciąg jest mniejszy niż 5
10    if (N < 5) {
11        return {};
12    }
13    //zmienne
14    const int M = 5;
15    std::vector<int> wynik;
16    //główna pętla algorytmu - przeszukiwanie wszystkich możliwych
podciągów
17    for(int i = 0; i < N - 4; i++) {
18        //wewnętrzna pętla algorytmu - szukanie spełnienia warunku
19        for(int j = i; j < N - 4; j++) {
20            //pętla iteracyjna po elementach podciągu
21            for(int k = 0; k < M; k++) {
22                //sprawdzenie warunku: suma elementów na pozycjach
nieparzystych > suma pozostałych
23                if(ciag[i+1] + ciag[i+3] > ciag[i] + ciag[i+2] + ciag
24                [i+4]) {
25                    //przepisanie znalezionego podciągu do tablicy
wyników
26                    for(int m = 0; m < M; m++) {
27                        wynik.push_back(ciag[i + m]);
28                    }
29                    //przerwanie pętli wewnętrznych po znalezieniu
podciągu
30                    j = N;
31                    k = M;
32                    break;
33                }
34            }
35        }
36        //zwrócenie liczby znalezionych podciągów
37        return wynik;
38    }
39
40    std::vector<int> podciag_wersja_2(int ciag[], int N) {
41        //program kończy się, jeśli tablica ciąg jest mniejszy niż 5
```

```

42     if (N < 5) {
43         return {};
44     }
45     // zmienne
46     const int M = 5;
47     int Temp[M];
48     std::vector<int> wynik;
49     //główna pętla algorytmu - przeszukiwanie wszystkich możliwych
    podciągów
50     for(int i = 0; i < N - 4; i++) {
51         //pętla wewnętrzna - przypisuje wartości do tablicy Temp
52         for(int j = 0; j < M; j++) {
53             Temp[j] = ciag[i + j];
54         }
55         //sprawdzenie warunku: suma elementów na pozycjach
    nieparzystych > suma pozostałych
56         if(Temp[1] + Temp[3] > Temp[0] + Temp[2] + Temp[4]) {
57             //pętla wewnętrzna - przypisująca z tablicy Temp wyniki
    do tablicy Wyniki
58             for(int k = 0; k < M; k++) {
59                 wynik.push_back(Temp[k]);
60             }
61         }
62     }
63     //zwrócenie liczby znalezionych podciągów
64     return wynik;
65 }
66
67 void wczytaj_dane(int ciag[], int N) {
68     //wczytuje dane z pliku
69     std::ifstream file("dane.txt");
70     if (file.is_open()) {
71         for (int i = 0; i < N; i++) {
72             file >> ciag[i];
73         }
74         file.close();
75     }
76 }
77
78 void generuj_dane(int ciag[], int N) {
79     //generuje dane poprzez bibliotekę random
80     std::srand(std::time(NULL));
81     for (int i = 0; i < N; i++) {
82         ciag[i] = rand() % 150 + 1; // rand() % (max - min + 1) +
    min
83     }
84 }
85
86 void wyswietl_dane(int tablica[], int N) {
87     //wyświetla dane początkowe (wygenerowane)
88     std::cout << "Ciag wejsciowy: ";

```

```

89     for (int i = 0; i < N; i++) {
90         std::cout << tablica[i] << " ";
91     }
92     std::cout << std::endl;
93 }
94
95 void wyswietl_wynik(std::vector<int> wynik) {
96     //wyswietla podciągi które zostały znalezione
97     if(!wynik.empty()) {
98         int k{0};
99         for (int i = 0; i < wynik.size() / 5; i++) {
100             std::cout << "[";
101             for (int j = 0; j < 5; j++) {
102                 std::cout << wynik[j+k] << " ";
103             }
104             std::cout << "]";
105             if (i < wynik.size() / 5 - 1) std::cout << ", ";
106             k += 5;
107         }
108         std::cout << std::endl;
109     }
110     else {
111         std::cout << "Ciag jest za krotki lub nie ma zadnego
112         spelnionego ciagu\n";
113     }
114 }
115
116 void zapis_do_pliku(std::vector<int> wynik, const std::string&
117     filename) {
118     //funkcja do zapisywania wyników do pliku
119     std::ofstream file(filename);
120     if (file.is_open()) {
121         if(!wynik.empty()) {
122             int k{0};
123             for (int i = 0; i < wynik.size() / 5; i++) {
124                 file << "[";
125                 for (int j = 0; j < 5; j++) {
126                     file << wynik[j+k] << " ";
127                 }
128                 file << "]";
129                 if (i < wynik.size() / 5 - 1) file << ", ";
130                 k += 5;
131             }
132         }
133     }
134
135     struct Wynik {
136         std::vector<int> p1;
137         std::vector<int> p2;
138     };

```



```

138
139 Wynik test_zestawow(std::vector<int> tab, int N) {
140     //funkcja do niewygodnych testów
141     std::vector<int> wynik = podciag(tab.data(), N);
142     std::vector<int> wynik2 = podciag_wersja_2(tab.data(), N);
143     return {wynik, wynik2};
144 }
145
146 void test_niewygodnych_zestawow() {
147     //funkcja związana z niewygodnymi testami
148     int tab1[] = {1, 3, 2, 3};
149     int tab2[] = {7, 7, 7, 7, 7};
150     int tab3[] = {1, 130, 1, 9, 11, 6, 1, 1, 1, 3, 1};
151     int tab4[] = {1, 100, 1, 100, 1, 2, 2, 2, 2};
152     int tab5[] = {5, 200, 5, 200, 5, 10, 10, 10, 10};
153
154     struct Test {
155         int* tablica;
156         int rozmiar;
157     };
158
159     Test testy[] = {
160         {tab1, sizeof(tab1)/sizeof(tab1[0])},
161         {tab2, sizeof(tab2)/sizeof(tab2[0])},
162         {tab3, sizeof(tab3)/sizeof(tab3[0])},
163         {tab4, sizeof(tab4)/sizeof(tab4[0])},
164         {tab5, sizeof(tab5)/sizeof(tab5[0])}
165     };
166
167     // Wykonanie wszystkich testów
168     for (int i = 0; i < 5; i++) {
169         std::cout << "\n=== TEST " << (i+1) << " ===" << std::endl;
170         std::cout << "Wejście: ";
171         std::cout << std::endl;
172         wyswietl_dane(testy[i].tablica, testy[i].rozmiar);
173
174         Wynik wynik = test_zestawow(
175             std::vector<int>(testy[i].tablica, testy[i].tablica +
176             testy[i].rozmiar),
177             testy[i].rozmiar
178         );
179
180         std::cout << std::endl;
181         std::cout << "Algorytm 1: ";
182         wyswietl_wynik(wynik.p1);
183         std::cout << "Algorytm 2: ";
184         wyswietl_wynik(wynik.p2);
185     }
186
187 void test_wydajnosci() {

```

```

188 //Utworzenie zmiennych startowych
189 int N[] = {2500, 5000, 10000, 20000, 30000, 40000, 50000, 60000,
190 70000, 80000, 90000, 100000, 120000, 140000, 160000, 180000,
191 200000, 250000, 300000, 350000, 400000, 450000, 500000};
192 int liczba_testow = sizeof(N)/sizeof(N[0]);
193 double* czas1 = new double[liczba_testow];
194 double* czas2 = new double[liczba_testow];
195
196 //Generowanie danych i wykonywanie testów
197 for(int i = 0; i < liczba_testow; i++) {
198     std::vector<int> ciag_wejscowy(N[i]);
199     std::vector<int> wynik(N[i]);
200
201     //Generowanie danych
202     generuj_dane(ciag_wejscowy.data(), N[i]);
203     //wyświetlanie danych
204     // wyswietl_dane(ciag_wejscowy.data(), N[i]);
205
206     //Rozpoczęcie głównego algorytmu
207     clock_t start1 = clock();
208     std::vector<int> wynik1 = podciag(ciag_wejscowy.data(), N[i
209 ]]);
210     clock_t stop1 = clock();
211     czas1[i] = (double)(stop1 - start1) / CLOCKS_PER_SEC;
212
213     //Rozpoczęcie głównego algorytmu drugiego
214     clock_t start2 = clock();
215     std::vector<int> wynik2 = podciag_wersja_2(ciag_wejscowy.
216 data(), N[i]);
217     clock_t stop2 = clock();
218     czas2[i] = (double)(stop2 - start2) / CLOCKS_PER_SEC;
219
220 }
221 //Wyświetlanie w konsoli czasu algorytmów
222 std::cout << "L.p.  n    t1[s]    t2[s]" << "\n";
223 for(int i = 0; i < liczba_testow; i++) {
224     std::cout << std::fixed << std::setprecision(6) << i+1 << "
225 " << N[i] << "    " << czas1[i] << "    " << czas2[i] << '\n';
226 }
227
228 std::string filename = "wyniki_wydajnosci.txt";
229 std::ofstream file;
230 file.open("wyniki_wydajnosci.txt");
231 file << "L.p.  n    t1[s]    t2[s]" << "\n";
232 for(int i = 0; i < liczba_testow; i++) {
233     file << std::fixed << std::setprecision(6) << i+1 << "    " <<
234 N[i] << "    " << czas1[i] << "    " << czas2[i] << '\n';
235 }
236 file.close();
237
238 delete[] czas1;

```

```
233     delete[] czas2;  
234 }  
235  
236 int main () {  
237  
238     return 0;  
239 }
```

Listing 1: Plik źródłowy: src/main.cpp